



COMPTE RENDU ELECTRONIQUE NUMÉRIQUE AVANCÉE

2023 - 2024
Master IESE

TP 3 Circuit logique programmable

Réalisé par:
Abaiji Mohamed
Encadré par:
Pr. Ahaitouf

- L'objectif de cette expérience était de programmer une carte PLD 16V8 à l'aide du langage ABEL.

- **Définition de la Carte GAL 16V8 :**

- **Définition :**

Une GAL (*Generic Array Logic*) 16V8 est un type de circuit logique programmable, souvent appelé PLD (*Programmable Logic Device*). Elle est composée de 16 entrées et 8 sorties et est programmable pour réaliser des fonctions logiques spécifiques. La dénomination "16V8" a une signification précise :

16 fait référence au nombre d'entrées disponibles sur la GAL.

8 indique le nombre de sorties qu'elle peut générer.

La programmation de ces entrées et sorties permet à la GAL 16V8 d'implémenter des fonctions logiques personnalisées en interconnectant les éléments logiques internes, tels que des logiques combinatoires et séquentielles, elle offre une flexibilité pour créer des fonctions logiques spécifiques à partir de ses entrées et sorties.

- **Architecture :**

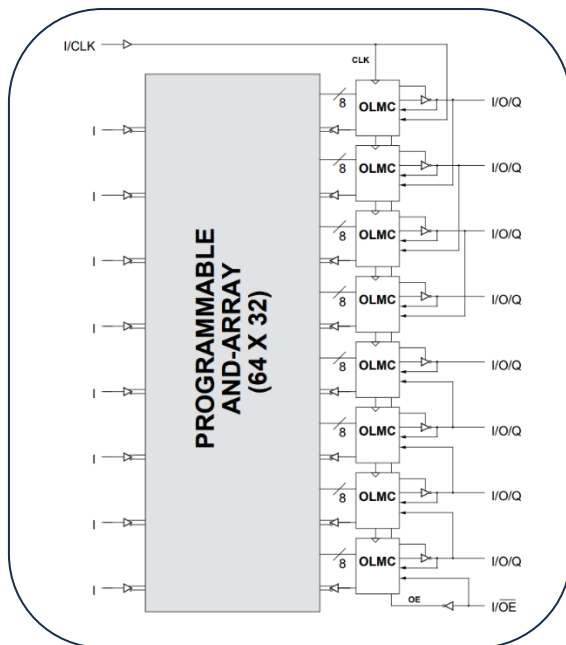
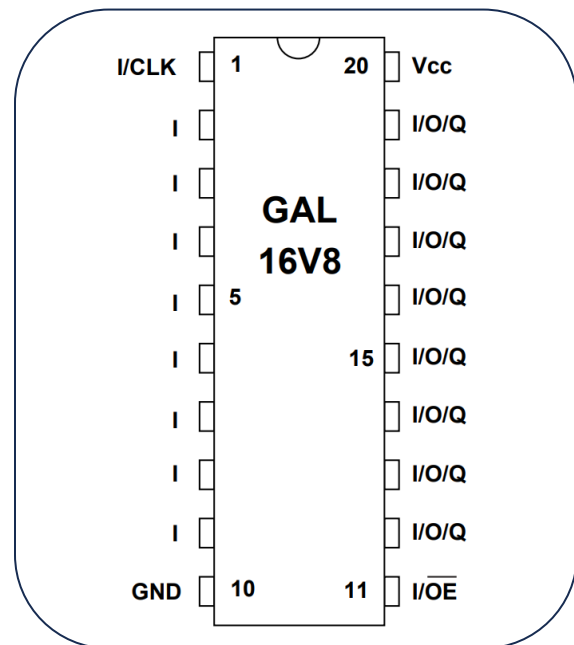


Schéma Bloc Fonctionnel



Configuration des broches

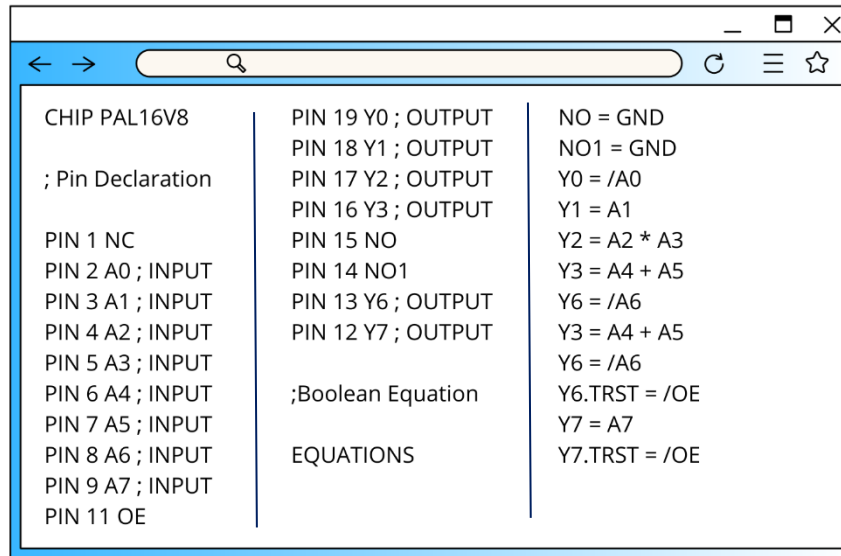
- **Programmation :**

Le langage ABEL est un choix pour programmer les cartes GAL. Il simplifie la création de circuits personnalisés en utilisant une syntaxe logique, Abel est un langage plus simple peut être utilisé pour décrire le comportement d'un système sous diverses formes, notamment des équations logiques, des tables de vérité et des diagrammes d'état. Le compilateur Abel permet de simuler des conceptions et de les implémenter dans des dispositifs logiques programmables.

- **Manipulation 1 : Programmation des portes logiques :**
 - **Les étapes pour charger un programme dans une carte PLD 16V8 :**

Écriture du Programme dans *WinIDE Studio-PLD* :

Utilisant *WinIDE Studio-PLD* pour écrire le programme. Nous allons définir la logique souhaitée en utilisant le langage de description matérielle *ABEL*.



Compilation du Programme :

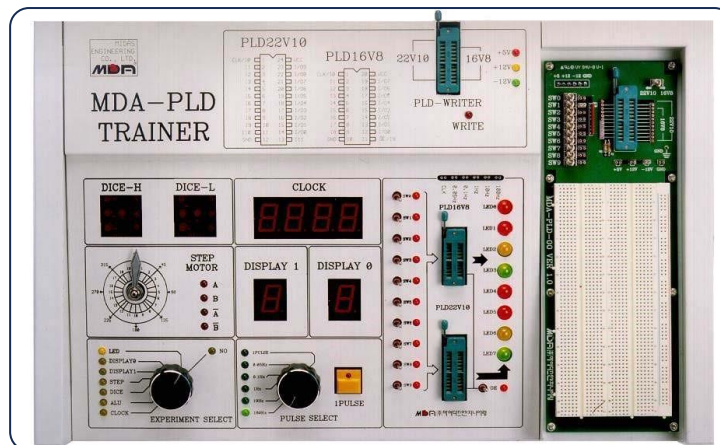
Une fois que nous avons écrit le programme dans *WinIDE Studio-PLD*, nous allons utiliser un compilateur PLD pour traduire le code en un fichier binaire exécutable.

Chargement du Programme dans la GAL 16V8 :

Connectons la carte PLD 16V8 à notre ordinateur via le câble. Chargez le fichier **.JEDEK** généré lors de l'étape précédente dans la carte PLD. Cela configure la carte avec le programme que nous avons créé.

Insertion de la Carte dans le MDA PLD Trainer :

Insérez la carte PLD 16V8 programmée dans le MDA PLD Trainer.



Test du Programme :

Allumons le *MDA PLD Trainer* tester le comportement de la carte PLD. Nous devrions pouvoir vérifier que la carte fonctionne conformément à la logique que nous avons programmée. Nous avons utilisé des entrées (*SWITCHS*) et des sorties (*LED*) pour observer le comportement du circuit.

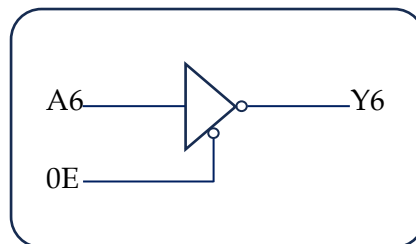
○ Vérification :

Nous assurons que les portes logiques programmées fonctionnent correctement.

Tampon trois états :

A6	OE	Y6
0	0	1
1	0	0
0	1	X
1	1	X

Tableau de vérité



Symbole

Lorsque l'entrée OE est au niveau bas, le tampon se comporte. Si OE est au niveau haut, la sortie est isolée de l'entrée

Fonctionnement

• Manipulation 2 : Programmation de logique combinatoire (Multiplexeur) :

Écriture du Programme dans *WinIDE Studio-PLD* :

```
CHIP MUX PAL16V8
;PIN Declarations
PIN 2..5 I[0..3]
PIN 6 SO
PIN 7 S1
PIN 12..18 NO[0..6]
PIN 19 Y

EQUATIONS

Y = /S1*/S0*I[0] + S1*/S0*I[1] + /S1*S0*I[2] + S1*S0*I[3]

NO[0]=GND
NO[1]=GND
NO[2]=GND
NO[3]=GND
NO[4]=GND
NO[5]=GND
NO[6]=GND
```

Vérification :

Le circuit permet de choisir l'une des quatre entrées de données en fonction des signaux de sélection (S0 et S1) et de diriger cette entrée sélectionnée vers la sortie.

- Manipulation 3 : Programmation des logique séquentielle (Stop Watch) :

CHIP STOP PAL16V8

; PIN Declarations

PIN 2 CLR
PIN 16..19 Q[0..3]
PIN 12..15 Q[4..7]

EQUATIONS

IF CLR = 0 THEN
BEGIN
Q[0..3] := #D0
Q[4..7] := #D0

END

ELSE
BEGIN

CASE (Qf0..3))
BEGIN
0: BEGIN
Q[0.3] := #D1
END
1: BEGIN
Q[0..3] := #D2
END
2: BEGIN
Q[0..3] := #D3
END
3: BEGIN
Q[0..3] := #D4
END
4: BEGIN
Q[0.3] := #D5
END
5: BEGIN
Q[0..3] := #D6
END
6: BEGIN
Q[0..3] := #D7
END
7: BEGIN
Q[0..3] := #D8

END

8: BEGIN
Q[0..3] := #D9
END

9: BEGIN
Q[0..3] := #D0
END

END

; CASE

IF Q[0..3] = #D9 THEN
BEGIN

CASE (Q[4..7])
BEGIN

0: BEGIN
Q[4..7] := #D1
END

1: BEGIN
Q[4..7] := #D2
END

2: BEGIN
Q[4..7] := #D3
END

3: BEGIN
Q[4..7] := #D4
END

4: BEGIN
Q[4..7] := #D5
END

5: BEGIN
Q[4..7] := #D6
END

6: BEGIN
Q[4..7] := #D7
END

7: BEGIN
Q[4..7] := #D8
END

8: BEGIN
Q[4..7] := #D9
END

9: BEGIN
Q[4..7] := #D0

```

END
END
ELSE
BEGIN
  Q[4] := Q[4]
  Q[5] := Q[5]
  Q[6] := Q[6]
  Q[7] := Q[7]
END
END

```



- **Analyse :**

Déclaration : **PIN Declarations**

Le circuit a une broche d'entrée CLR (Clear) et des broches de sortie pour les chiffres des unités (Q[0..3]) et les chiffres des dizaines (Q[4..7]).

Partie 1 : **IF** ...

Si CLR est à 0 (niveau bas), le circuit efface les valeurs stockées dans les registres Q[0..7] en les mettant à 0 (représenté par #D0).

END

ELSE

Partie 2 : **BEGIN**

Cette section gère l'incrémement des chiffres des unités (Q[0..3]). Elle utilise une structure CASE pour conditionner la valeur actuelle de Q[0..3], et incrémente correctement ces chiffres.

Exemple : lorsque la valeur est 2, Q[0..3] est mise à jour pour afficher le chiffre 3 (#D3).

Si Q [0..3] atteint 9, il est remis à 0 pour représenter 10.

... **END** :

Partie 3 : **IF**

Cette section gère l'incrémement des chiffres des dizaines (Q[4..7]) lorsque les chiffres des unités (Q[0..3]) atteignent 9. Elle utilise une structure CASE

similaire pour examiner la valeur actuelle de Q[4..7] , et incrémente ces chiffres correctement en fonction de leur état actuel.

... **END**

ELSE

Partie 4 : **BEGIN**

Cette section gère les cas où ni les chiffres des unités ni des dizaines n'atteignent 9.

Si c'est le cas, toutes les valeurs de Q[4..7] restent inchangées.

END

END

CHIP STOP PAL22V10

; PIN DECLARATIONS

PIN 2..9 IN[7..0]

correspondent aux sorties d'un autre GAL16V8

PIN 10 CLR

PIN 20..23

PIN 16..9 Q[4..7]

PIN 14 POINT1

PIN 15 POINT0

EQUATIONS

POINT0 = GND

POINT1 = VCC

IF CLR = 0 THEN

 BEGIN

 Q[0..3] := #D0

 Q[4..7] := #D0

 END

ELSE BEGIN

IF IN[7]*IN[4]*IN[3]*IN[0]=1 THEN **si IN[7..0] est égal à 99**

 BEGIN

 0: BEGIN

 Q[0..3] := #D1

incrémenter les unités

 ...

 END

```

8: BEGIN
    Q[0..3] := #D9
    END
9: BEGIN
    Q[0..3] := #D0
END
END
ELSE
BEGIN
    Q[4] := Q[4]
    Q[5] := Q[5]
    Q[6] := Q[6]
    Q[7] := Q[7]
    END
END
IF IN[7]*IN[4]*IN[3]*IN[0]*Q[3]*Q[0] = 1 THEN
    sorties d'un GAL16V8 = 99 et les
    unites de GAL22V10 = 9.

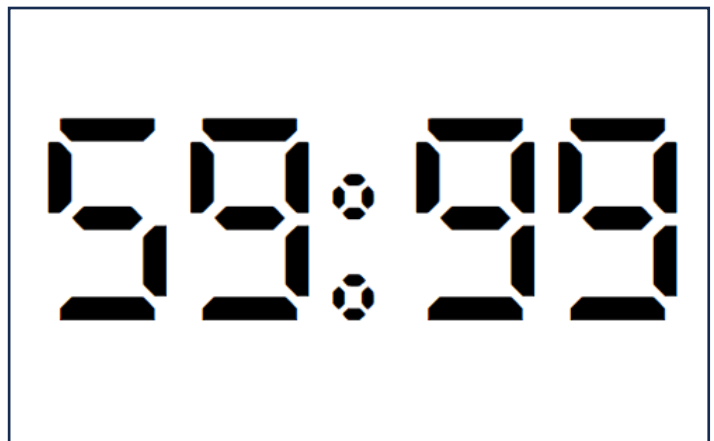
    BEGIN
    CASE (Q[4..7])
        BEGIN
        0: BEGIN
            Q[4..7] := #D1
            END

            ...

        4: BEGIN
            Q[4..7] := #D5
            END
        5: BEGIN
            Q[4..7] := #D0
            END
        END
    END
END
ELSE
BEGIN
    Q[4] := Q[4]
    Q[5] := Q[5]
    Q[6] := Q[6]
    Q[7] := Q[7]
    END
END
END

```

déclencher l'ajustement des chiffres des dizaines à 5



À une fréquence de 100 Hz, le compteur affiche les secondes (s) et les millisecondes (ms).